ITMO-444

Weather Advisor TwoTier APIs

Jay Han
jhan38@hawk.illinoistech.edu

Checklist

Output of ls -R lab06 | head -40, showing the layout from Part 0.

```
Jay  ~/Desktop/Repo/ITMO-444  main ±  ls -R lab06 | head -40
advisor-app
app_smoke_test.py
create_database.sh
create_weather_stack.sh
db_outputs.env
delete_database.sh
delete_weather_stack.sh
large_test.txt
previous
stack_outputs.env
test_image.jpg
test.txt
test2.txt
testing
weather-app

lab06/advisor-app:
app.js
node_modules
package-lock.json
package.json

lab06/advisor-app/node_modules:
@aws
@aws-sdk
@smithy
accepts
array-flatten
body-parser
bowser
bytes
call-bind-apply-helpers
call-bound
content-disposition
content-type
cookie
cookie-signature
debug
depd
destroy
```

Output of the NWS calls from Part A and one sentence on why two upstream calls are needed.

1. **ZIP code to coordinate**

```
Jay  ~/Desktop/Repo/ITMO-444  main ±  curl -s https://api.zippopotam.u
s/us/60616 \
  | python3 -m json.tool
{
    "country": "United States",
    "country abbreviation": "US",
    "post code": "60616",
    "places": [
        {
            "place name": "Chicago",
            "longitude": "-87.6306",
            "latitude": "41.8426",
            "state": "Illinois",
            "state abbreviation": "IL"
        }
    ]
}
```

```
Jay  ~/Desktop/Repo/ITMO-444  main ±  curl -s -o /dev/null \
  -w "unknown zip status: %{http_code}\n" \
  https://api.zippopotam.us/us/00000
unknown zip status: 404
```

2. **NWS grid**

```
Jay  ~/Desktop/Repo/ITMO-444  main ±  FORECAST_URL=$(curl -s \
  -H "User-Agent: $UA" \
  "https://api.weather.gov/points/41.8458,-87.6272" \
  | python3 -c \
    "import sys,json; print(json.load(sys.stdin)['properties']['forecast'])")

echo "$FORECAST_URL"
https://api.weather.gov/gridpoints/LOT/76,72/forecast
```

3. **Call forecast**

```
Jay 🍎  ~/Desktop/Repo/ITMO-444  main ±  curl -s \
  -H "User-Agent: $UA" \
  "$FORECAST_URL" \
  | python3 -m json.tool \
  | head -40
{
    "@context": [
        "https://geojson.org/geojson-ld/geojson-context.jsonld",
        {
            "@version": "1.1",
            "wx": "https://api.weather.gov/ontology#",
            "geo": "http://www.opengis.net/ont/geosparql#",
            "unit": "http://codes.wmo.int/common/unit/",
            "@vocab": "https://api.weather.gov/ontology#"
        }
    ],
    "type": "Feature",
    "geometry": {
        "type": "Polygon",
        "coordinates": [
            [
                [
                    -87.6047,
                    41.8435
                ],
                [
                    -87.6031,
                    41.8653
                ],
                [
                    -87.6324,
                    41.8665
                ],
                [
                    -87.634,
                    41.8447
                ],
                [
                    -87.6047,
                    41.8435
                ]
            ]
        ]
    },
    "properties": {
```

4. **One sentence**
   The reason behind having two upstream calls is that the National Weather Service cannot utilize ZIP codes. The first upstream call converts the ZIP code to latitude and

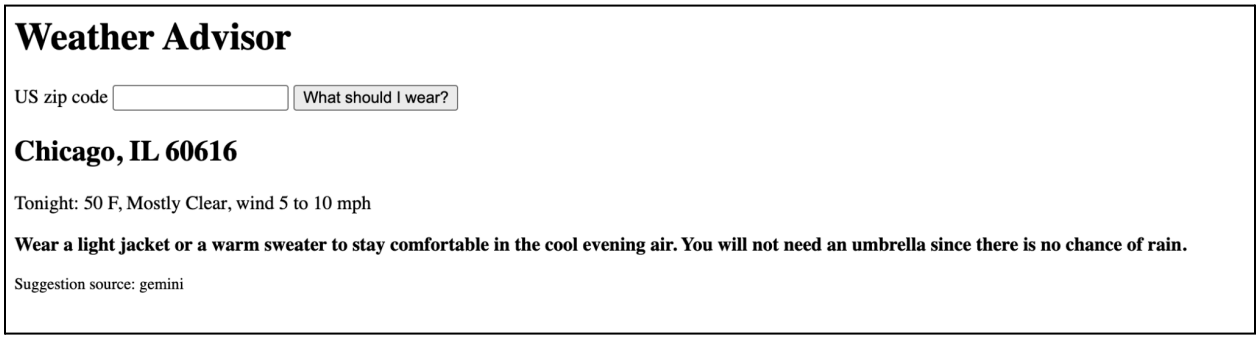
longitude, and then the second is done to obtain the NWS forecast URL and forecast using the latitude and longitude coordinates

Output of create_database.sh and the Step 4 output showing retention 0, not public, single-AZ, and empty snapshot and automated backup lists.

```
✘ Jay  ~/Desktop/Repo/ITMO-444/lab06  main ±  cd /Users/jayhan/Desktop/Repo/ITMO-444/lab06
./create_database.sh | tee create_database_output.txt
== 1. Default VPC and subnets in both AZs ==
== 2. Database security group (port 5432, from the app security group only) ==
Database security group already exists, skipping.
== 3. DB subnet group ==
DB subnet group already exists, skipping.
== 4. Database password (random, stored as a SecureString, never printed) ==
Password parameter already exists, keeping it.
== 5. The PostgreSQL instance (free tier size, no backups) ==
Database itmo-444-544-weather-db already exists, skipping.
Waiting for the database to become available (usually 5 to 10 minutes)...
Done. Database endpoint: itmo-444-544-weather-db.c1q6k4ewksjb.us-east-2.rds.amazonaws.com
Automated backups are OFF (retention 0) and no final snapshot will be taken on delete.
Next: ./create_weather_stack.sh. When testing is finished: ./delete_weather_stack.sh then ./delete_database.sh
```

```
-----------------------------------------
|          DescribeDBInstances          |
+-------------------------+---------------+
|  BackupRetentionDays    |  0            |
|  Class                  |  db.t4g.micro |
|  Encrypted              |  True         |
|  Engine                 |  postgres     |
|  MultiAZ                |  False        |
|  Public                 |  False        |
+-------------------------+---------------+
```

Screenshot of the advisor page showing a suggestion for a zip code, and the JSON from /advice?zip=... for the same zip.




```
Jay  ~/Desktop/Repo/ITMO-444/lab06  main ±  cd /Users/jayhan/Desktop/Repo/ITMO-444/lab06

export $(grep -v '^#' .env | xargs)
source stack_outputs.env

curl -s \
  "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" \
  | python3 -m json.tool
{
    "zip": "60616",
    "city": "Chicago",
    "state": "IL",
    "forecast": {
        "name": "Tonight",
        "isDaytime": false,
        "windSpeed": "5 to 10 mph",
        "temperature": 50,
        "shortForecast": "Mostly Clear",
        "temperatureUnit": "F",
        "precipitationChance": 0
    },
    "latitude": "41.8426",
    "longitude": "-87.6306",
    "cache": "hit",
    "advice": "Wear a warm jacket and long pants.",
    "source": "fallback",
    "model": null
}
```

Output of app_smoke_test.py with all nine checks passing.

```
Jay  ~/Desktop/Repo/ITMO-444/lab06  main ±  cd /Users/jayhan/Desktop/Repo/ITMO-444/lab06/testing
python3 app_smoke_test.py
PASS  weather tier returns a forecast  status 200
PASS  a repeat call is answered from the database (cache hit)  cache=hit
PASS  history lists the saved lookups  status 200, count=9
PASS  weather tier rejects a bad zip with 400  status 400
PASS  weather tier returns 404 for an unknown zip  status 404
PASS  advisor tier returns clothing advice  status 200, 0.7s
        source=gemini model=gemini-3.1-flash-lite
        advice: Wear a light jacket over a long-sleeved shirt to stay comfortable in the cool evening air. You will not need an umbrella since there is no chance of precipitation.
PASS  advice came from Gemini (not the fallback)  fallback means the key, quota or model name needs a look
PASS  advisor tier rejects a bad zip with 400  status 400
PASS  advisor home page loads  status 200

9 of 9 checks passed
```

Step1: a few lines of the weather journal, with a sentence explaining one weather_success line.

```
Oct 29 23:45:46 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:46.279Z","event":"weather_rejected","reason":"zip_not_found","zip":"00000"}
Oct 29 23:45:46 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:46.384Z","event":"weather_request","zip":"60616"}
Oct 29 23:45:46 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:46.385Z","event":"weather_success","zip":"60616","cache":"hit","ms":1}
Oct 29 23:45:52 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:52.777Z","event":"weather_request","zip":"60616"}
Oct 29 23:45:52 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:52.779Z","event":"weather_success","zip":"60616","cache":"hit","ms":2}
Oct 29 23:45:52 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:52.843Z","event":"weather_request","zip":"60616"}
Oct 29 23:45:52 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:52.845Z","event":"weather_success","zip":"60616","cache":"hit","ms":2}
Oct 29 23:45:52 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:52.975Z","event":"weather_rejected","reason":"invalid_zip"}
Oct 29 23:45:53 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:53.053Z","event":"weather_request","zip":"00000"}
Oct 29 23:45:53 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:53.074Z","event":"weather_rejected","reason":"zip_not_found","zip":"00000"}
Oct 29 23:45:53 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:53.218Z","event":"weather_request","zip":"60616"}
Oct 29 23:45:53 ip-172-31-14-71.us-east-2.compute.internal node[11358]: {"ts":"2026-10-29T23:45:53.220Z","event":"weather_success","zip":"60616","cache":"hit","ms":2}
```

Step2: the /history output and the psql query output.


```
{
    "count": 9,
    "lookups": [
        {
            "id": "9",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Mostly Clear",
            "temperature": "50",
            "created_at": "2026-10-29T23:39:33.934Z"
        },
```

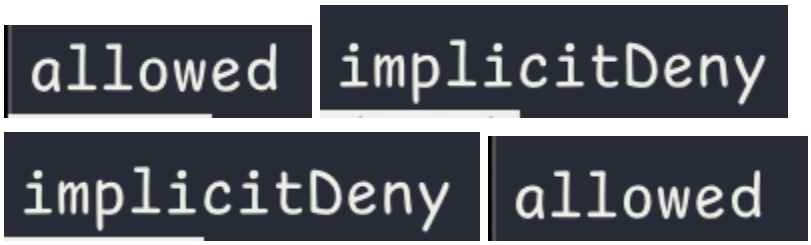

```
 id |  zip  |          created_at
----+-------+-------------------------------
  9 | 60616 | 2026-10-29 23:39:33.934631
  8 | 60616 | 2026-10-29 21:57:56.866597
  7 | 98107 | 2026-10-29 21:49:05.83483
  6 | 97201 | 2026-10-29 21:48:03.460763
  5 | 98101 | 2026-10-29 21:47:57.083426
(5 rows)

 saved_lookups
---------------
             9
(1 row)
```

Step3: the miss and hit timings, and your two sentences.

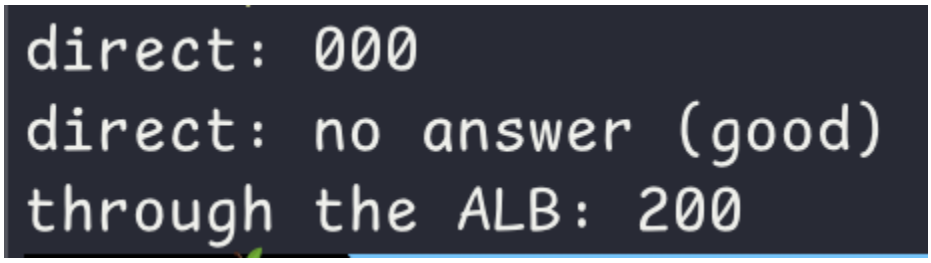
```
request 1: status=200 time=0.216466s
cache = miss | city = Atlanta
request 2: status=200 time=0.060130s
cache = hit | city = Atlanta
```

Step5: the IAM simulator results for both roles and both secrets, and your sentence about the advisor tier.



Hackers can get hold of the Gemini API key simply by breaching the advisor tier. However, it is not possible for them to know the database password since the advisor role prohibits access to that parameter.

Step6: the direct and through-the-ALB results, and your sentence about security group sources.



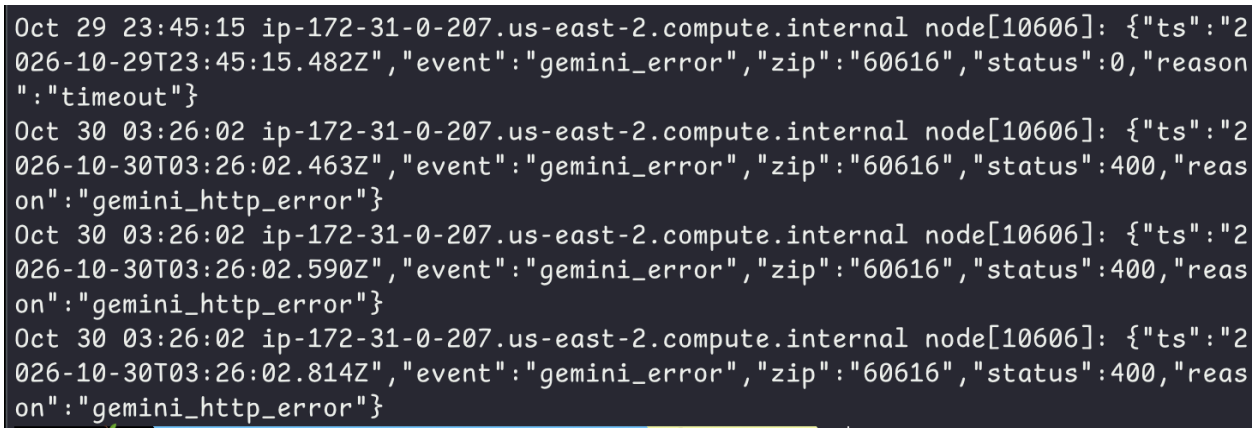
With the security group's source rule, resources that are associated with the same security group can send and receive traffic no matter what their current IP address is, whereas CIDR rules control traffic only according to the designated IP address range.

Step7: the output while the database was cut off, the db_error log line, and your sentence about the 30-second pause.

```
Jay 🍎  ~/Desktop/Repo/ITMO-444/lab06  main ±  curl -s \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" \
  | python3 -c \
    "import sys,json; d=json.load(sys.stdin); print(d['city'], 'cache =', d['cac
he'])"
Chicago cache = unavailable
Jay 🍎  ~/Desktop/Repo/ITMO-444/lab06  main ±  curl -s -o /dev/null \
  -w "history while the database is cut off: %{http_code}\n" \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history"
history while the database is cut off: 503
Jay 🍎  ~/Desktop/Repo/ITMO-444/lab06  main ±  ssh -i "$PEM_PATH" \
  ec2-user@"$(instance_ip weather-app)" \
  "sudo journalctl -u weather -n 40 --no-pager | grep db_error | tail -3"
Oct 30 00:02:59 ip-172-31-14-71.us-east-2.compute.internal node[12379]: {"ts":"2
026-10-30T00:02:59.408Z","event":"db_error","stage":"read","message":"Connection
 terminated due to connection timeout"}
Oct 30 03:19:47 ip-172-31-14-71.us-east-2.compute.internal node[18284]: {"ts":"2
026-10-30T03:19:47.128Z","event":"db_error","stage":"read","message":"Connection
 terminated due to connection timeout"}
```

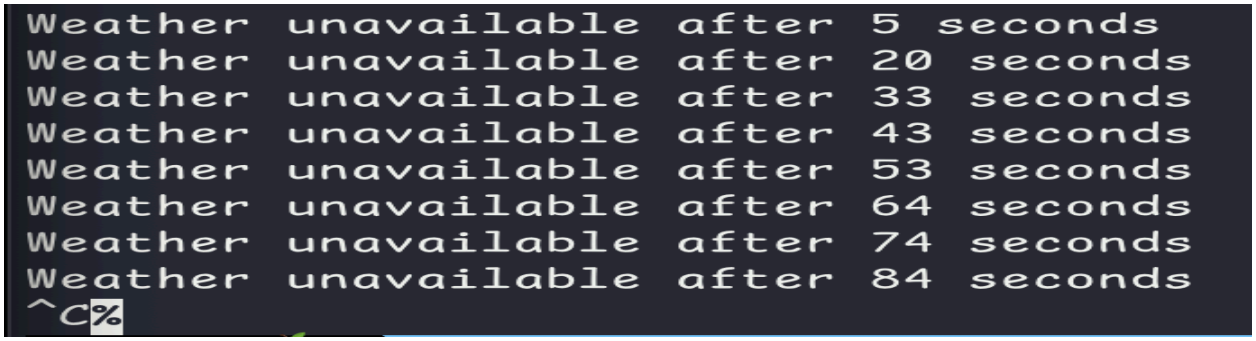
The 30-second pause makes sure that no request would continue to try connecting to the database; otherwise every request would keep waiting for another timeout and burden the missing database with unnecessary load.

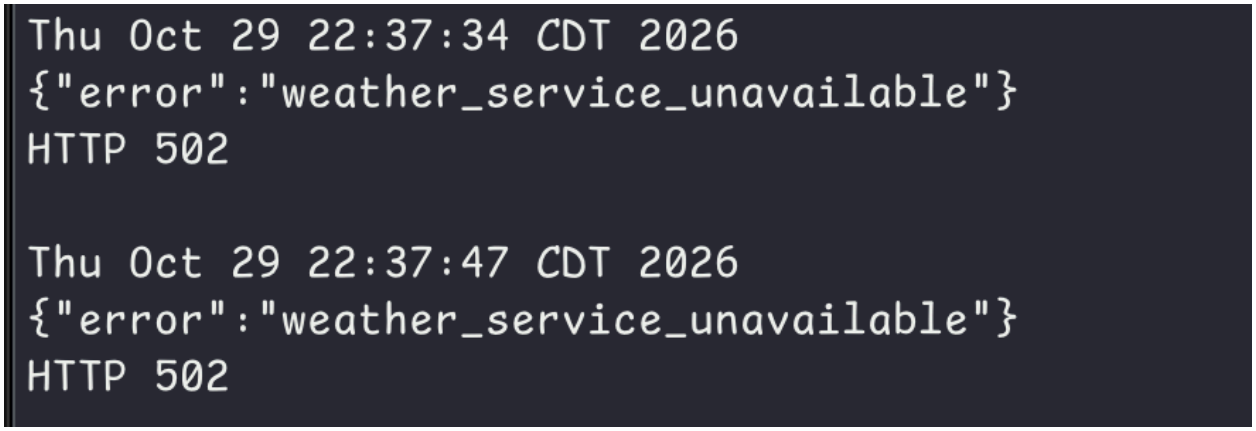
Step 8: the fallback answers, the gemini_error journal lines, and your sentence about the 30-second pause.



Silent fallback keeps the service available, but it can hide a persistent API key, model, or quota failure if nobody monitors the gemini_error logs.

Step 9: the recovery time you measured, what the advisor returned during the outage, and your sentence about where state should live.





Application state should live in a managed database rather than on an EC2 instance, because Auto Scaling instances are replaceable and their local data disappears when they are terminated.

One paragraph on why the secrets are in SSM and each is ready by one role, instead of in .env or the user-data.

The key for the Gemini API and the password for the database have been stored as encrypted SSM SecureString parameters rather than being kept in .env files or EC2 user-data. Secrets are stored in .env files, which may sometimes be unintentionally committed to Git, whereas user-data can be accessed by software running on the instance or by anyone who has access to the launch template. Each EC2 role is allowed to read only one specific parameter according to the hierarchy: the one responsible for the weather part reads the password for the database, and the one which gives advice reads the key for Gemini. Such a least-privilege approach minimizes the impact of the compromise of one level, as a secret cannot be retrieved from another level.

One paragraph on turning backups off: what it saved, what it costs you, and when you would never do it.

By setting the RDS backup retention period to zero and deleting the database without creating a final snapshot, the expenses related to storage and backup of this temporary lab were minimized, and the cleanup process was made easier. The downside of this decision is that no deleted or corrupted data can be recovered, and point-in-time recovery becomes unavailable. This is okay for disposable lab data, but I would never turn off backups for production, financial, medical, academic, client, or any other important data. In actual production, automated backups should be enabled with an appropriate retention period, final snapshots created, the recovery procedures tested, and potential cross-region or cross-account copies created where applicable.

One paragraph on what data you sent to Gemini and why the free tier makes that matter.

The advisor sent Gemini only the provided zip code, city, and country, forecast period, temperature, weather conditions, wind speed, and precipitation. They did not send the name, account information, database history, IP address, database password, or anything else with personal information. This is important as Gemini is a third-party service and the free tier might have different limits, privacy policy, data handling, and guarantees from a paid or institutional-level account. I would check the current terms and seek institutional approval or user agreement before using it with real users.

Answers to the Section 15.1 review questions.

1. Where else could each leak from (shell history, process lists, logs, screenshots, Git)?

=> Secrets could leak through shell history, process lists, logs, screenshots, Git, or exposed configuration files.

2. What would change if both roles had ssm:* on *?

=> With ssm:* on *, either compromised role could read, modify, or delete every SSM secret.

3. Why is the ALB open to the internet but the internet but the instance and the database are not?

=> The ALB must accept users' requests, while EC2 and RDS should accept traffic only from trusted security groups.

4. What can an advisor instance do that it should not be able to do? How would you fix it?

=> The advisor can reach the database port. Separate the weather and advisor security groups and allow only the weather group.

5. What would you create instead, and which privileges would it need?

=> Create a dedicated application user with only SELECT and INSERT permissions on the lookups table.

6. What does the certificate check protect against that TLS without it does not?

=> Certificate verification confirms the server's identity and helps prevent man-in-the-middle attacks.

7. Show one line of the app and say what would go wrong if it built SQL by joining strings

=> VALUES ($1, $2) keeps input separate from SQL; string concatenation could let attackers modify the query.

8. What would you lose if this database held real data, and what would you configure instead?

=> Real data would be permanently lost. Enable automated backups, retention, and final snapshots.

9. Who decides when stored data may be destroyed in a real organization?

=> Data owners and the organization's legal, privacy, and retention policies should determine when data is deleted.

10. What could an attacker gain if the page printed raw model output?

=> Raw AI output could inject HTML or JavaScript and cause cross-site scripting.

11. What would you have to change before using a paid or school-owned Gemini account for real users?

=> Obtain privacy and institutional approval, review provider terms, and minimize or anonymize user data.

12. How could someone use up your quota, and what could you add?

=> Attackers could exhaust the quota with automated requests; add rate limiting, WAF rules, authentication, and alerts.

13. Why is it important that the response says source: fallback or cache: unavailable?

=> fallback and cache: unavailable reveal degraded service instead of falsely presenting the response as normal.

Output of the Step 10 verification commands showing no database instance, snapshot, automated backup, load balancer, or lab role remains.

